



Eidgenössische Technische Hochschule Zürich
Swiss Federal Institute of Technology Zurich

Parallel Programming
Assignment 6: Task Parallelism
Spring Semester 2026

Assigned on: **25.03.2026**

Due by: **(Wednesday Exercise) 30.03.2026**
(Friday Exercise) 01.04.2026

Overview

This week's assignment is about Task Parallelism. Task parallelism emphasizes the distributed (parallelized) nature of the processing (i.e., threads), as opposed to the data (data parallelism). We will use the fork/join framework to implement task parallelism. The fork/join framework helps to take advantage of multiple processors. It is designed for work that can be broken into smaller pieces recursively. The goal is to use all the available processing power to enhance the performance of your application.

Getting Prepared

- Fetch the new assignment from Gitlab by running:

```
git fetch upstream  
git merge upstream/main
```

You should now see the new assignment appear in your local directory.

- Open the project in IntelliJ IDEA: Click on *File* in the top-menu, then select *Open*. Select the folder of the new assignment.

1 Sorting

In this exercise you are asked to implement the merge-sort algorithm using task parallelism. The merge-sort algorithm sorts an array by partitioning it in smaller arrays. Once the size of the arrays becomes 1 or 2, they are trivially sorted. Sorted sub-arrays are combined by *merging* them.

The simplest task in merge-sort is the trivial sort of small arrays. This is the following:

```
if (array.length == 1) {  
    // An array of size 1 is implicitly sorted  
    return array;  
}  
else if (array.length == 2) {  
    // Re-arrange elements if they are not in proper order  
    if (array[0] > array[1]) {  
        temp = array[0];  
        array[0] = array[1];  
        array[1] = temp;  
    }  
    return array;  
}
```

Merging two sorted arrays into a bigger sorted array is also a simple procedure. Consider the following two sorted arrays:

```
array1: [5, 11, 12, 18, 20]  
array2: [2, 4, 7, 11, 16, 23, 28]
```

The resulting merged array is computed by comparing the head of the two arrays. The smallest one is removed and it is appended at the end of the output. In this case, the merged output is:

```
result: [2, 4, 5, 7, 11, 11, 12, 16, 18, 20, 23, 28]
```

The code to merge two ordered arrays is already provided to you in `ethz.ch.pp.assignment6.util.ArrayUtils`. Further, we also provide a sequential merge-sort in `ethz.ch.pp.assignment6.mergeSort.MergeSortSingle`.

Your task is to implement the task-parallel version of merge-sort.

Task A: The code you downloaded and imported contains relevant parts in `MergeSortMulti` that you are expected to implement. For merge-sort with task parallelism, the following is one of the possible strategies to divide the work:

```
sort(input) {  
  
    if (input.length <= 2) {  
  
        // Execute the simple task locally  
        return simpleSort(input);  
  
    } else {  
  
        // Split the input in two parts by forking to two tasks  
  
        fork(firstPart);  
        fork(secondPart);  
  
        waitForResults();  
  
        // Join results  
        return merge(firstPartResults, secondPartResults);  
  
    }  
  
}
```

Implement the strategy outlined above using the `ForkJoinPool` and `RecursiveAction` Java APIs.

Task B: Verify that the multi-threaded merge-sort computes the correct results by running the provided set of test-cases.

2 Longest Sequence

In this exercise our goal is to find the longest sequence of the same consecutive number in an input sequence of numbers. For example, we show the longest sequences for a given input below:

$$[1, 9, 4, 3, 3, 8, 7, 7, 7, 0] \xrightarrow{\text{longest sequence}} [7, 7, 7]_{\text{start:6}}^{\text{end:8}}$$

Where the longest sequence is formed by three consecutive numbers 7 that start at index 6 and end at index 8. For all non-empty inputs the longest sequence always contains at least one element. In case of multiple sequences having the same length we always return the one with smaller starting index. We illustrate both of these cases with following two examples:

$$\begin{aligned} [0, 1, 2, 3, 4, 5, 6, 7, 8, 9] &\xrightarrow{\text{longest sequence}} [0]_{\text{start:0}}^{\text{end:0}} \\ [1, 1, 0, 0] &\xrightarrow{\text{longest sequence}} [1, 1]_{\text{start:0}}^{\text{end:1}} \end{aligned}$$

We provide a sequential version that returns the longest sequence of the same consecutive number in `LongestCommonSequence` class. You may assume that the input array has always at least one element.

Task A: Implement a task parallel version that computes longest sequence using the fork/join framework in `LongestCommonSequenceMulti` class. Start with a cutoff set to value 2. Note that in this task we cannot simply split the input array into two partitions whose results can be computed independently and combined afterwards. For example, if we would split input `[1, 3, 3, 2]` into two parts `[1, 3]` and `[3, 2]` we would miss the longest sequence `[3, 3]` as it is on the boundary. However, for many inputs (such as `[3, 3, 1, 2]`) we can safely split them and compute the results independently. Make sure your implementation handles this case and successfully parallelizes the computation without producing incorrect results. Use the provided set of test cases to verify your implementation.

Task B: Improve the performance of the implementation from Task 1 by choosing a more appropriate cutoff value. Compare the performance to the sequential version. Note that the computation performed in the base case (e.g., comparing that two array values are the same) is very simple and fast. To make the task more compute-intensive, use more expensive comparison (e.g., `Math.exp(i) == Math.exp(j)` instead of `i == j`).

Submission

You need to submit your code and reports by using our submission system. This step will be the same for all of the subsequent exercises, but we will explain it in detail here so you can familiarize yourself with the system.

To submit your code using the terminal, follow the instructions below:

- In the terminal, navigate to your personal repository.
- Add your changes:
`git add .` # this command adds all changes in the git repo
- Commit your changes. In your commit message, make sure to include the [Submission] tag.
`git commit -m "[Submission] ..."`
- Push your changes!
`git push`

To submit your code using IntelliJ, follow the instructions below:

- Press the “Commit” button (found on the left hand toolbar)
- Select all the files you want changed during the assignment.
- Write a descriptive commit message. Make sure to include the [Submission] tag.
- Press “Commit and Push”

After this step, you can check your repository on the GitLab website in your browser. Make sure that you see your changes there!

Troubleshooting

Git Issues

In rare cases, you might get an error like this:

```
Can't connect to any URI
https://gitlab.inf.ethz.ch/COURSE-PPROG26/pprog26-<username>.git
(https://gitlab.inf.ethz.ch/COURSE-PPROG26/pprog26-<username>.git:
https://gitlab.inf.ethz.ch/COURSE-PPROG26/pprog26-<username>.git/info/refs?
service=git-receive-pack not found)
```

It means that no repository was created for your username yet. If that is the case, please contact your teaching assistant. The teaching assistant will be able to create the repository location for you. After that, you should be able to submit your work by following the instructions above.

IntelliJ / Java Issues

If you have trouble running the Java programs in IntelliJ, check the following:

- You have marked the `src` directory as a “Source Root”. Right click on the folder `src`, select “Mark Directory As >Source Root”.
- You have installed an updated JDK. We recommend `openjdk-25`. You can check this under “File >Project Structure >SDK”